

FFT & IFFT 完整学习笔记

摆渡

2026 年 1 月 25 日

目录

1	核心目标与应用价值	2
1.1	核心功能	2
1.2	应用意义	2
2	预备知识（必记）	2
2.1	多项式的两种表示方法	2
2.2	n 次单位根的关键性质	2
2.2.1	定义	2
2.2.2	两大核心性质	2
3	FFT 详解（系数 $\rightarrow$ 点值）	3
3.1	Python 代码实现	3
3.2	核心数学推导	4
3.2.1	多项式分治拆分	4
3.2.2	蝴蝶操作的数学原型	4
3.2.3	线性变换本质（拓展）	4
4	IFFT 详解（点值 $\rightarrow$ 系数）	5
4.1	Python 代码实现	5
4.2	核心数学推导	6
4.2.1	逆变换的本质：FFT 矩阵的逆	6
4.2.2	逆旋转因子的由来	6
5	FFT 与 IFFT 核心差异对比	6
6	关键思想总结	7
7	大一适配补充说明	7

1 核心目标与应用价值

1.1 核心功能

- **FFT**（快速傅里叶变换）：将多项式的 **系数表示** 转化为 **点值表示**，时间复杂度 $O(n \log n)$ 。
- **IFFT**（逆快速傅里叶变换）：将多项式的 **点值表示** 转回 **系数表示**，时间复杂度 $O(n \log n)$ 。

1.2 应用意义

传统多项式乘法（系数直接两两相乘）的时间复杂度为 $O(n^2)$ ，通过 FFT + IFFT 的组合，可将复杂度降至 $O(n \log n)$ ，大幅提升大规模数据下的计算效率。

2 预备知识（必记）

2.1 多项式的两种表示方法

表示方式	具体形式	加法复杂度	乘法复杂度	核心优势
系数表示	$P(x) = p_0 + p_1x + p_2x^2 + \cdots + p_{n-1}x^{n-1}$	$O(n)$	$O(n^2)$	直观易懂，便于
点值表示	n 个点 $(x_0, P(x_0)), \dots, (x_{n-1}, P(x_{n-1}))$	$O(n)$	$O(n)$	乘法高效，对应点

表 1: 多项式的两种表示方法对比

2.2 n 次单位根的关键性质

2.2.1 定义

n 次单位根是满足 $\omega_n^n = 1$ 的复数，其数学表达式为：

$$\omega_n = e^{\frac{2\pi i}{n}} = \cos\left(\frac{2\pi}{n}\right) + i \sin\left(\frac{2\pi}{n}\right)$$

其中 i 为虚数单位，几何意义是 ** 复平面上把单位圆 n 等分的点 **。

2.2.2 两大核心性质

1. **对称性**： $\omega_n^{j+\frac{n}{2}} = -\omega_n^j$ ，后半部分单位根是前半部分的相反数。
2. **周期性**： $\omega_n^k = \omega_{\frac{n}{2}}^{\frac{2k}{2}}$ ，单位根平方后次数减半，取值不变。

3 FFT 详解（系数 $\rightarrow$ 点值）

3.1 Python 代码实现

Listing 1: FFT 递归实现代码

```
1 import math
2
3 def FFT(P):
4     """
5     快速傅里叶变换：系数表示转点值表示
6     :param P: 系数列表 [p0, p1, ..., pn-1], n需为2的幂
7     :return: 点值列表 [P(1), P(-1), ..., P((-1)n-1)]
8     """
9     n = len(P)
10    if n == 1:
11        return P # 基线条件：常数多项式点值=系数
12
13    # 生成n次单位根（旋转因子）
14    omega = math.e ** (2 * math.pi * 1j / n)
15
16    # 拆分偶数次项和奇数次项子多项式
17    Pe = P[::2] # 偶数下标系数
18    Po = P[1::2] # 奇数下标系数
19
20    # 递归计算子多项式的FFT
21    ye = FFT(Pe)
22    yo = FFT(Po)
23
24    # 蝴蝶操作合并结果
25    y = [0] * n
26    for j in range(n // 2):
27        y[j] = ye[j] + (omega ** j) * yo[j]
28        y[j + n//2] = ye[j] - (omega ** j) * yo[j]
29
30    return y
```

3.2 核心数学推导

3.2.1 多项式分治拆分

对于长度为 n (n 为 2 的幂) 的多项式 $P(x)$, 按系数下标奇偶性拆分为两个子多项式:

$$\begin{aligned} P_e(x) &= p_0 + p_2x + p_4x^2 + \cdots + p_{n-2}x^{\frac{n}{2}-1} \\ P_o(x) &= p_1 + p_3x + p_5x^2 + \cdots + p_{n-1}x^{\frac{n}{2}-1} \end{aligned}$$

原多项式可表示为:

$$P(x) = P_e(x^2) + x \cdot P_o(x^2)$$

该式将计算 $P(x)$ 的问题, 转化为计算两个 $\frac{n}{2}$ 次多项式的问题, ** 规模减半 **。

3.2.2 蝴蝶操作的数学原型

选择 n 次单位根 ω_n^j 和 $\omega_n^{j+\frac{n}{2}}$ 作为求值点:

1. 代入 ω_n^j , 利用周期性 $(\omega_n^j)^2 = \omega_{\frac{n}{2}}^j$:

$$P(\omega_n^j) = P_e(\omega_{\frac{n}{2}}^j) + \omega_n^j \cdot P_o(\omega_{\frac{n}{2}}^j)$$

2. 代入 $\omega_n^{j+\frac{n}{2}}$, 利用对称性 $\omega_n^{j+\frac{n}{2}} = -\omega_n^j$:

$$P(\omega_n^{j+\frac{n}{2}}) = P_e(\omega_{\frac{n}{2}}^j) - \omega_n^j \cdot P_o(\omega_{\frac{n}{2}}^j)$$

通过一次子问题计算, 可同时得到两个点值, 避免重复计算, 这就是 ** 蝴蝶操作的核心 **。

3.2.3 线性变换本质 (拓展)

FFT 可表示为矩阵与向量的乘法:

$$\vec{y} = F_n \cdot \vec{p}$$

其中:

- $\vec{p} = [p_0, p_1, \dots, p_{n-1}]^T$ 为系数向量;
- $\vec{y} = [P(\omega_n^0), P(\omega_n^1), \dots, P(\omega_n^{n-1})]^T$ 为点值向量;
- F_n 为 n 阶 FFT 矩阵, 元素满足 $F_n[j, k] = \omega_n^{jk}$ 。

4 IFFT 详解（点值 $\rightarrow$ 系数）

4.1 Python 代码实现

Listing 2: IFFT 递归实现代码

```
1 import math
2
3 def IFFT(P):
4     """
5     逆快速傅里叶变换：点值表示转系数表示
6     :param P: 点值列表 [P( ), P(  $\omega^n$  ), ..., P(  $\omega^{n-1}$  )], n 需为 2 的幂
7     :return: 系数列表 [p0, p1, ..., pn-1]
8     """
9     n = len(P)
10    if n == 1:
11        return P # 基线条件：单点值对应系数
12
13    # 生成逆旋转因子（逆单位根 + 1/n 缩放）
14    omega = (1 / n) * (math.e ** (-2 * math.pi * 1j / n))
15
16    # 拆分偶数位置和奇数位置点值
17    Pe = P[::2]
18    Po = P[1::2]
19
20    # 递归计算子点值的 IFFT
21    ye = IFFT(Pe)
22    yo = IFFT(Po)
23
24    # 蝴蝶操作合并结果（与 FFT 完全一致）
25    y = [0] * n
26    for j in range(n // 2):
27        y[j] = ye[j] + (omega ** j) * yo[j]
28        y[j + n//2] = ye[j] - (omega ** j) * yo[j]
29
30    return y
```

4.2 核心数学推导

4.2.1 逆变换的本质：FFT 矩阵的逆

FFT 矩阵 F_n 是 ** 对称矩阵 **，且满足共轭转置性质：

$$F_n \cdot F_n^* = n \cdot I_n$$

其中 F_n^* 是 F_n 的共轭转置矩阵， I_n 是 n 阶单位矩阵。两边同除以 n ，得逆矩阵公式：

$$F_n^{-1} = \frac{1}{n} \cdot F_n^*$$

系数向量 $\vec{p}$ 是点值向量 $\vec{y}$ 与逆矩阵的乘积：

$$\vec{p} = F_n^{-1} \cdot \vec{y} = \frac{1}{n} \cdot F_n^* \cdot \vec{y}$$

4.2.2 逆旋转因子的由来

F_n^* 的元素为 $\overline{\omega_n^{jk}} = (\omega_n^*)^{jk}$ ，其中 $\omega_n^* = e^{-\frac{2\pi i}{n}} = \omega_n^{-1}$ （逆单位根）。因此逆变换公式可写为：

$$p_j = \frac{1}{n} \sum_{k=0}^{n-1} y_k \cdot (\omega_n^{-1})^{jk}$$

对比 FFT 公式，逆变换仅需 ** 两处调整 **：

1. 旋转因子从 ω_n 改为 ω_n^{-1} （共轭）；
2. 结果整体乘以缩放因子 $\frac{1}{n}$ 。

5 FFT 与 IFFT 核心差异对比

对比维度	FFT（正变换）	IFFT（逆变换）
核心目标	系数 $\rightarrow$ 点值	点值 $\rightarrow$ 系数
旋转因子	$\omega = e^{\frac{2\pi i}{n}}$	$\omega = \frac{1}{n} \cdot e^{-\frac{2\pi i}{n}}$
数学本质	$\vec{y} = F_n \cdot \vec{p}$	$\vec{p} = \frac{1}{n} F_n^* \cdot \vec{y}$
代码差异	仅旋转因子定义行不同，分治 + 蝴蝶框架完全复用	

表 2: FFT 与 IFFT 核心差异

6 关键思想总结

1. **分治是效率核心**：将 $O(n^2)$ 复杂度的问题拆分为 $O(n \log n)$ ，与归并排序思想相通。
2. **数学支撑编程**：单位根的对称性、周期性是蝴蝶操作的前提，算法的优雅性依赖数学性质。
3. **代码复用典范**：仅修改核心参数（旋转因子）即可实现正逆变换，体现「通用框架 + 参数化」思维。
4. **适用前提**：多项式长度需为 2 的幂，工程中可通过 **** 补零 **** 满足该条件。

7 大一适配补充说明

- **复数运算**：Python 原生支持复数类型，`1j` 代表虚数单位 i ，无需手动分离实部/虚部。
- **精度修正**：IFFT 结果可能存在微小虚部误差，可通过 `[round(x.real, 6) for x in res]` 取实部并舍入。
- **迭代版拓展**：递归版易发生栈溢出，工程常用 **** 位逆序重排 **** 的迭代版，核心思想与递归版一致。